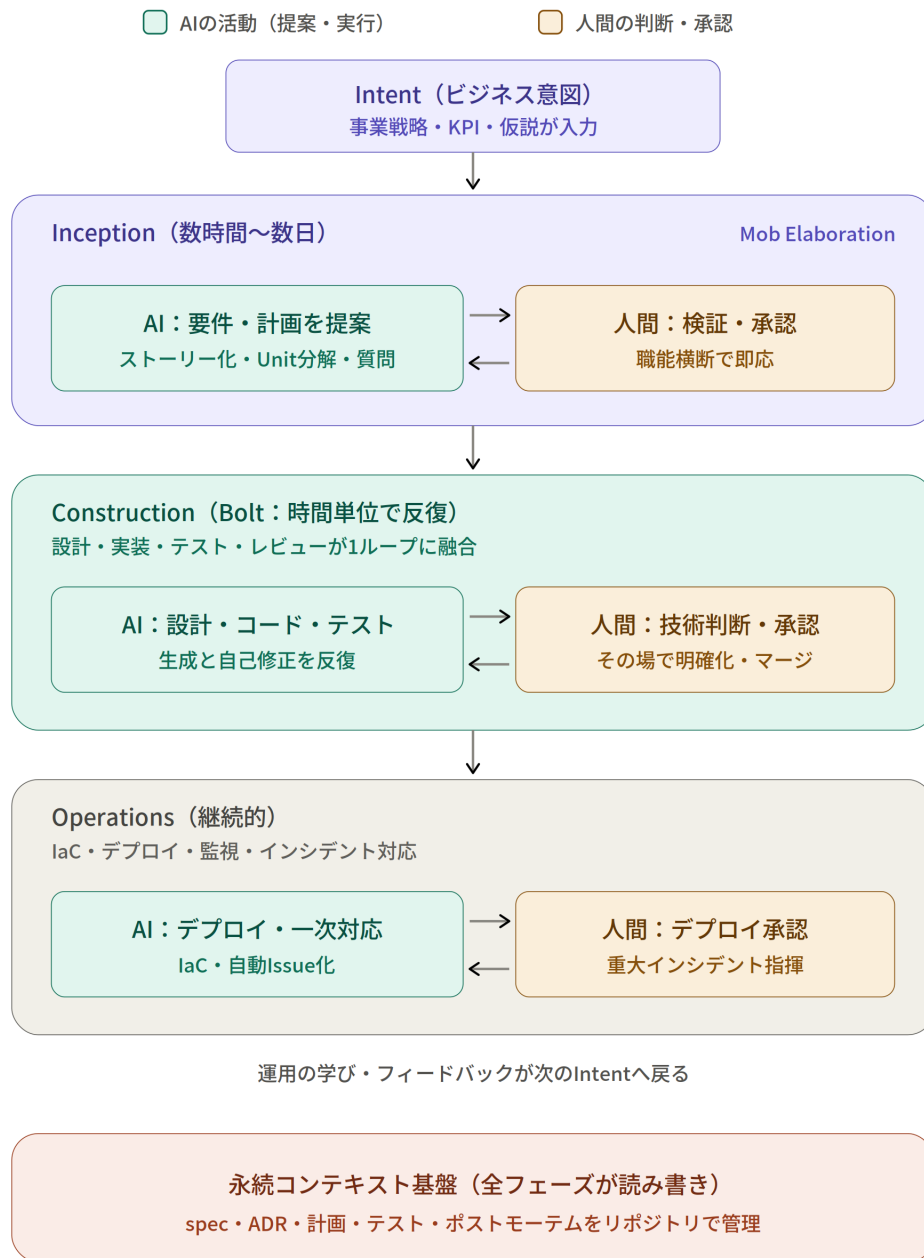


AI-DLC 実践リファレンス

AI駆動開発ライフサイクル



4つの設計原則

1. 主導権の逆転	AIが計画を提案し、質問し、人間が承認する。あくまで人間の知見を中心に進行し、人間のチェックを高頻度・軽量に繰り返す。
2. 4つの承認ゲート	仕様承認・アーキテクチャ承認 (Inception内) / マージ承認 (Construction内) / デプロイ承認 (Operations内)。人間の時間はこの4点に濃く配置し、それ以外はAIに委譲する。
3. ドキュメント徹底	spec・ADR・計画・テストをリポジトリに永続化。各工程のAI出力が次工程のAI入力になる(図の最下段)。「AIが読めない場所の決定」を作らない。
4. 儀式的置き換え	Sprint→Bolt (時間～日単位)、Epic→Unit of Work、非同期レビュー→モブ (Mob Elaboration / Mob Construction)。見積もり工程は事実上消える。

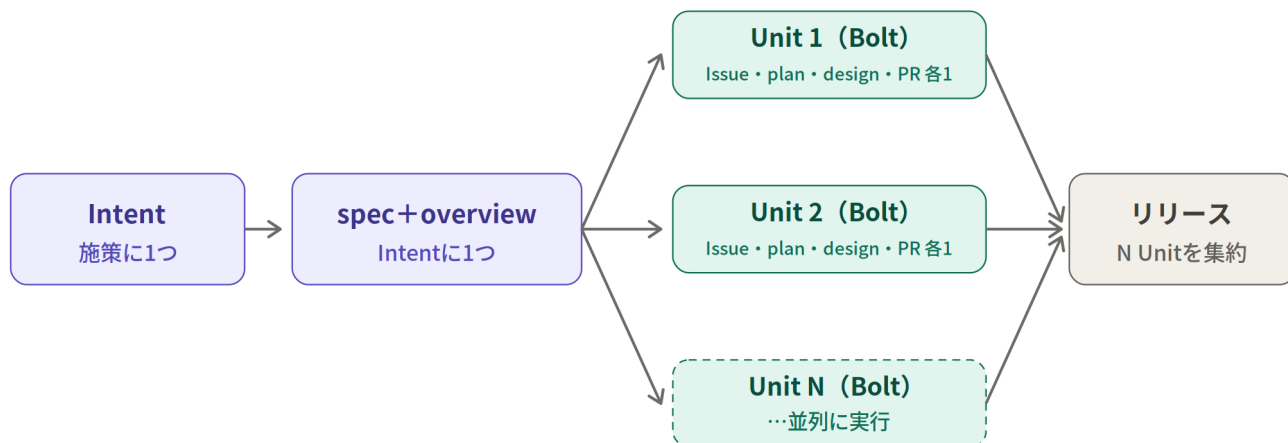
設計は「二層」に分割する

基本設計 Inception	技術選定・サービス境界・データ基盤・認証方式など、間違えると高くつく決定。エージェントが高速に動き出す前に人間が承認する。ADRの対象はこちら。
Unit詳細設計 Construction	ドメインモデル・コンポーネント・API詳細。実装の手戻りコストが崩壊したため事前に固める理由が消え、Boltループ内でjust-in-timeに行う。

従来フローとの対応（全12ステップ＝4×3フェーズ）

Inception	Intent (戦略) / 仕様策定 / アーキテクチャ方針 / 計画・タスク分解	ステップ1～4
Construction	Unit詳細設計 (JIT) / 実装 / コードレビュー / テスト・QA	ステップ5～8
Operations	CI/CD / デプロイ / 監視 / フィードバック分析	ステップ9～12

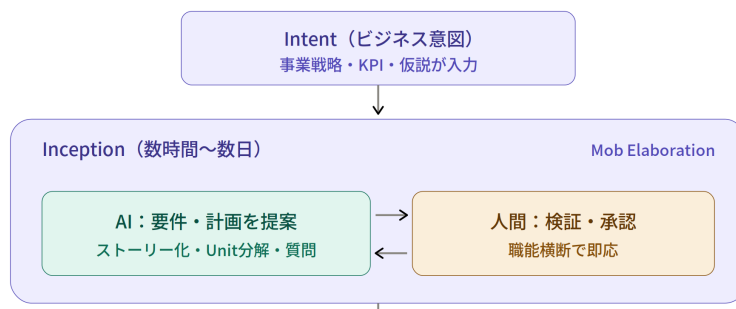
1つのIntentがN個のUnit of Workに分解され、Unitごとに独立のBoltループが並列に回る



分岐点＝ステップ4（計画・タスク分解） 合流点＝ステップ10（デプロイ・リリース）

成果物	単位（何に対して1つか）	置き場所・形式
product.md	プロダクトに1つ（面・アクター・施策の索引）	docs/product/product.md
glossary.md（用語集）	プロダクトに1つ（ユビキタス言語）	docs/product/glossary.md
intent.md	施策に1つ	docs/<施策>/intent.md
spec.md	Intentに1つ（大規模なら分割）	docs/<施策>/spec.md
ADR	不可逆な決定ごとに1件（施策横断で参照・蓄積）	docs/adr/NNNN.md
overview.md（分解マップ）	施策に1つ：Unit一覧・依存・共有コンポーネント	docs/<施策>/overview.md
Issue（Unit of Work）	Unitごとに1件（Issue＝Unitの正）	GitHub Issues・Jira
plan.md（Unit実行プラン）	Unitごとに1つ（AIのplan出力を保存）	docs/<施策>/<unit>/plan.md
design.md	Unitごとに1つ	docs/<施策>/<unit>/design.md
PR	Unitごとに原則1件	GitHub
リリースノート	デプロイごとに1つ＝N Unitを集約	タグ・CHANGELOG（自動生成）
ポストモテム	インシデントごとに1件	docs/postmortems/

Inception — 意図から計画へ（ステップ1〜4／施策単位＝1つの話）



1. Intent（戦略・プロダクト方向性の確定）

ドキュメント	intent.md(施策に1つ:狙い・North Star KPI・仮説)／PRD／VoC分析レポート
ツール	Claude(ディープリサーチ・壁打ち)／Notion・Google Docs／Amplitude・Mixpanel
ポイント	事業判断と「何を作らないか」の決定は人間。AIには市場・競合・VoC(チケット・レビュー・議事録)の分析と、選択肢・根拠・反証の網羅を任せる。決定と理由は必ずテキスト化して保存(以降の全AIの前提コンテキストになる)。新施策はdocs/product/product.mdの施策マップに登録。

2. 仕様策定 ★フロー全体の最重要工程

ドキュメント	spec.md(Intentに1つ)／受け入れ基準(Gherkin: Given/When/Then)／UIモック
ツール	GitHub Spec Kit・Claude Code(対話で具体化)／v0(動くモックで検証:spec↔v0を往復)
ポイント	エッジケース・例外系・非機能要件はAIに網羅的に洗い出させる。受け入れ基準は機械可読に書き、そのままテスト生成の入力にする。仮説と成功指標もここに仕込む(→ステップ12で自動検証)。対象面(ユーザー画面/管理画面)を冒頭に1行明記し、用語はdocs/product/glossary.mdに従う。
人間ゲート	ゲート① 仕様承認 — 以降の自動化はすべてこのspecを正として動く。人間の時間を最も濃く投下する。

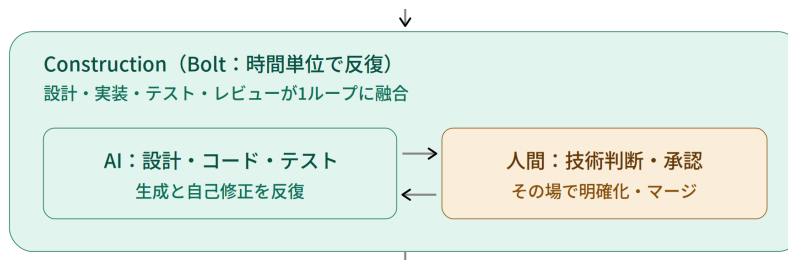
3. アーキテクチャ方針（不可逆な決定のみ）

ドキュメント	ADR(決定ごとに1件・リポジトリに蓄積)／システム構成図／技術選定メモ
ツール	Claude Code(既存コードベースを読ませ選択肢比較・影響範囲分析)／Mermaid
ポイント	ここで決めるのは「間違えると高くつく決定」のみ:技術選定・サービス境界・データ基盤・認証方式。詳細設計はConstructionへ先送り(JIT化)。AIにトレードオフ表を作らせ、人間が選び、理由をADRに確定。口頭決定もADR/Issueに残す。
人間ゲート	ゲート② アーキテクチャ承認 — 不可逆な決定に限定。Unitレベルの設計はConstructionのモブ内で軽量に承認。

4. 計画・タスク分解 — 「1→N」の分岐点

ドキュメント	overview.md(施策に1つ:Unit一覧・依存・共有コンポーネント)／GitHub Issue・Jiraチケット(Unitごとに1件)
ツール	GitHub Issues・Projects／Jira・Linear／n8n(Slack議論→仕様→Issue自動起票)
ポイント	詳細設計より先に分解する(AI-DLCの順序)。分解単位は設計モジュールではなく「ユーザー価値の垂直スライス」。ここから先はUnit 1個ごとの話になり、ConstructionはUnit単位の独立ループとして並列に回る。共有コンポーネントに触るUnitはoverview.mdでタグ付けし、設計承認を厚めに。対象面も同じタグ軸で扱う(例:[面:admin])。Issueには参照spec/ADR・完了条件を必須化。

Construction — Boltで回す実装ループ（ステップ5～8／Unitごとに1ループ）



5. Unit詳細設計（just-in-time）

ドキュメント	design.md(Unitごとに1つ)／Unit実行プラン(Unitごとに1つ:AIのplan出力を保存)／API仕様(OpenAPI)
ツール	Claude Code(Mob Constructionで対話設計・planモード)／Mermaid／OpenAPI
ポイント	設計はUnit着手時にBoltループ内で行う。AIが設計案を提示→チームがその場で明確化→即実装へ。直前のUnitで得た学びを反映できるため、事前一括設計より精度が高い。spec(ステップ2)とアーキテクチャ方針(ステップ3)が制約として効く。ADRに値する判断は都度docs/adr/へ。

6. 実装

ドキュメント	PR(Unitごとに原則1件)／CLAUDE.md(規約・前提)／.claude/(skills・commands・hooks)
ツール	Claude Code(サブエージェント・Hooks・GitHub Actions連携)／git worktreeで並列化
ポイント	Issue駆動: @claudeメンション→実装PRが自動で上がる形に。Hooksでlint・型・テストを強制(通らないPRは物理的に作れない状態)。分解済みなのでUnit単位でエージェントを並列実行できる。人間が直接書くのは中核アーキテクチャ・高リスク箇所のみ。

7. コードレビュー

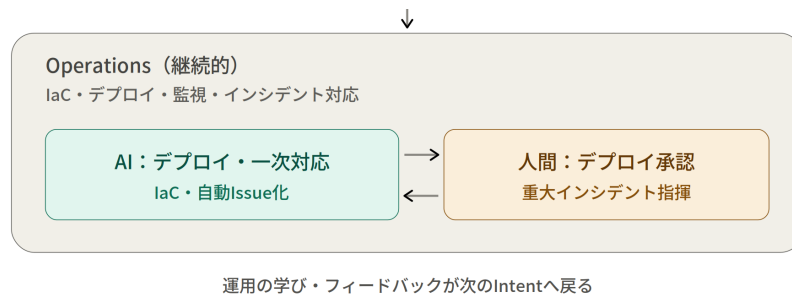
ドキュメント	AIレビューコメント／セキュリティ・静的解析レポート
ツール	CodeRabbit・Greptile・Claude Codeレビュー(一次)／CodeQL・Semgrep(脆弱性)
ポイント	AI一次レビュー×自動修正のループを人間なしで収束させてから人間に届ける。人間の観点は「spec適合と設計判断」のみ(構文・規約はAIと機械に任せる)。
人間ゲート	ゲート③ マージ承認 — 組織スリム化後も絶対に残す。Mob Constructionでその場承認に寄せると速い。

8. テスト・QA

ドキュメント	テスト設計書／E2Eコード／QAワークフロー定義(リスク分析→計画→設計)
ツール	Playwright(+MCPでエージェント自身が動作確認)／pytest・vitest
ポイント	ユニットテストは受け入れ基準(Gherkin)から自動生成し実装ループに内包。QAプロセス自体をタスク分割してAIワークフロー化。人間は探索的テストと受け入れ確認に集中。

Boltの感覚: ステップ5～8は「工程」ではなく、Unit 1個につき1回転するループ(図の往復矢印)。エージェントが設計案を出し、書き、テストし、自己修正してからPRが出てくるため、設計／実装／テスト／レビューの境界は実質溶ける。1 Unit=1 Bolt=数時間～1日が目安。

Operations — 継続的デリバリーと学習（ステップ9～12／N Unitを集約する）



9. CI/CD

ドキュメント	.github/workflows/(パイプライン定義)／失敗ログ→自動修正PR
ツール	GitHub Actions+Claude Code Action／flakyテスト検出ツール
ポイント	CI失敗ログをエージェントに渡し修正コミットまで自動化。CIの強さ=AI自律度の上限なので、ここへの投資を最優先にする。パイプライン定義の保守もAIに委譲。

10. デプロイ・リリース管理 — 「N→1」の合流点

ドキュメント	リリースノート(デプロイごとに1つ・N Unit分のPRを集約して自動生成)／フィーチャーフラグ設定／ロールバック基準書
ツール	LaunchDarkly・Unleash(フラグ)／Argo Rollouts(カナリア)／semantic-release
ポイント	並列に完了した複数Unitをここで集約して届ける。カナリア中のメトリクス比較→ロールバック判定までエージェント化し、人間の承認をワンクリックに軽量化。「承認後の張り付き」をなくす。
人間ゲート	ゲート④ デプロイ承認 — 本番反映の最終判断。基準(メトリクス閾値)を事前に文書化しておく。

11. 監視・運用・インシデント対応

ドキュメント	ランブック／ポストモータム(インシデントごとに1件・AIが草案)／自動起票されたIssue
ツール	Datadog・Sentry(AI機能)／PagerDuty／MCP連携(ログ・トレース・メトリクスをエージェントに接続)
ポイント	軽微な障害は「検知→Issue化→実装→承認→デプロイ」の自己修復ループへ(ステップ4以降の再利用:障害1件=Unit 1個として同じルールに乗せる)。重大インシデントの指揮と対外判断は人間。

12. フィードバック分析（ループを閉じる）

ドキュメント	仮説検証レポート(定期自動生成)／KPIダッシュボード
ツール	Amplitude・BigQuery+Claude／cron+エージェントで週次レポート
ポイント	spec.md(ステップ2)に仕込んだ仮説・成功指標とリリース後データを突合し「仮説は当たったか」を自動判定。学びを次のIntent(ステップ1)に還元してループを閉じる。

実践例 — 認証/認可の施策

上から順にフローを追う: Intent (施策の狙い) → spec (何を・なぜ) → overview (Nに分解) → Issue (Unitの契約書) → Unit実行プラン (作業の順序・動词的) → design (出来上がりの形・名詞的)。

① Intent

intent/auth.md (施策に1つ)

狙い: 認証まわりの問い合わせを減らし、離脱を下げる

North Star: 認証起因の離脱率

仮説: パスワード関連の問い合わせを50%削減できれば、離脱が改善する

② spec.md

docs/auth/spec.md (Intentに1つ)

対象面: user-web (ユーザー画面) … docs/product/product.md の面一覧に対応

1. サインアップ … (省略)

2. ログイン / セッション … (省略)

3. パスワード管理

3.1 パスワードポリシー

- 12文字以上、英大小・数字・記号のうち3種以上

3.2 パスワードリセット ← このUnitはここを切り出したもの

背景(WHY): ログイン不能による問い合わせがサポート工数の18%を占める

要件(WHAT):

- 登録済みメール宛にリセットリンクを送れること
- リンクは発行から30分で失効すること
- 失効後のアクセスには再発行を促すこと
- 新パスワードは 3.1 のポリシーを満たすこと

非機能: リセット要求は同一IPから5回/時に制限

対象外: SMSリセット / ソーシャルログイン連携

③ overview.md — 施策をN個のUnitに分解 (1→N)

docs/auth/overview.md (施策に1つ / ステップ4でAIが提案し人間が承認)

Unit一覧 (Intentを N個の垂直スライスに分解):

U0 reset_tokens 等の共通スキーマ 依存: なし (最初に単独で実施)

U1 パスワードリセット 依存: U0

U2 ログイン試行のレート制限 依存: U0 ※共有: 認証ミドルウェア

U3 リセットメールのテンプレート 依存: U1

U4 メール確認つきサインアップ 依存: U0

U5 セッション有効期限の見直し 依存: なし ※共有: 認証ミドルウェア

実施順序: U0 → (U1, U2, U4, U5 は並列) → U3

共有コンポーネント: 認証ミドルウェア = U2・U5 が変更 → 触れる Unit は設計承認を厚く

※ overviewは「どのUnitがあり、どの順で、どこが衝突するか」だけを持つ。以降は U1 の中身を追う。

実践例（続き） — Unit U1「パスワードリセット」の中身

④ Issue (Jira / GitHub) — このUnitの契約書。WHATと完了条件

[Unit] U1 パスワードリセット

参照: docs/auth/spec.md#3.2 , adr/0007-token-store.md

完了条件 (Given/When/Then):

- 登録済みメールにリセットリンクが届く
- リンクは30分で失効する / 失効後アクセス時は再発行を促す
- 新パスワードは既存ポリシー(3.1)を満たす

非対象: SMSリセット, ソーシャルログイン

状態: 未着手 → Boltループ中 → レビュー待ち → マージ済み

⑤ Unit実行プラン (plan出力) — 着手前の段取り。順序と検証手順

docs/auth/password-reset/plan.md (Unitごとに1つ / Claude Codeのplan出力を保存)

段取り:

1. reset_tokens テーブルのマイグレーション追加
2. TokenService(生成・失効ロジック) 実装 → 単体テスト
3. POST /auth/reset-request 実装 → 統合テスト
4. POST /auth/reset-confirm 実装 → 統合テスト
5. Mailer にフックしメール疎通を E2E で確認

依存: adr/0007 に従いトークン保管先は Redis

リスク: 既存ログインのレート制限と競合の可能性 → 手順3で確認

※ 使用技術の選定は書かない(ADR) / クラス構造は書かない(design)

⑥ design.md — 出来上がりの内部構造

docs/auth/password-reset/design.md (Unitごとに1つ)

コンポーネント: ResetController → TokenService → Mailer

データモデル:

reset_tokens(id, user_id, token_hash, expires_at, used_at)

API:

POST /auth/reset-request {email} → 202
POST /auth/reset-confirm {token, new_password} → 200 / 410(失効)

シーケンス: Controller → TokenService.issue → Mailer.send

補足：プロダクト前提 — docs/product/（施策の上位・常設）

「ユーザー画面」「管理画面」のような面(Surface)や、アクター・用語は、狙い・KPI・仮説を持たない常設の前提であり、最上位(Intentの上位)とする。面と施策は直交する(1つの施策が複数の面に着地しうる)。

```
# docs/product/product.md （プロダクトに1つ / 常設・低頻度更新）
```

面(Surface):

user-web ユーザー画面 ... 一般ユーザー向けWeb
admin-web 管理画面 ... 社内オペレーター向け(要admin権限)

アクター:

一般ユーザー ... user-web の利用者
オペレーター ... admin-web の利用者(社内)
システム ... バッチ・エージェント

施策マップ(索引):

auth 認証/認可 → docs/auth/
billing 請求 → docs/billing/

※ 施策の中身はここに書かない。docs/<施策>/ への索引に徹する。

```
# docs/product/glossary.md （プロダクトに1つ / ユビキタス言語）
```

面(Surface) : ユーザーが触れる恒常的な入口(user-web / admin-web)

オペレーター : admin-web を利用する社内ユーザー

Unit : 独立してリリース可能なユーザー価値の垂直スライス

リセットトークン : パスワード再設定用の短命トークン(30分TTL)

※ 並列に走るN Unitのエージェントが別々の語を発明しないための共通語彙。
spec・overview・design・実装のすべてがこの語彙に従う。

※ 面はディレクトリ軸ではなくタグ軸で扱う。docs/ の親は施策のまま崩さない(面ごとのフォルダは作らない)。

補足：ADR — 施策を横断する不可逆な決定

ADRは①～⑥のフローには乗らない。Intentごとを書くものではなく、施策を横断して蓄積される低頻度・不可逆の決定だから。実践例の Issue・plan・design が参照していた adr/0007 の実体がこれ：

adr/0007-token-store.md （不可逆な決定ごとに1件 / 施策横断で蓄積・参照）

タイトル：パスワードリセット等の短命トークンの保管先

状態：Accepted 日付：2026-02-10

文脈：

リセットトークンは「30分で失効」等のTTLを持ち、失効後は破棄したい。
永続化は不要だが、複数の Cloud Run インスタンス間で共有が必要。

決定：

トークンは PostgreSQL ではなく Memystore(Redis) に TTL 付きで保管する。
※ 採用スタック：Next.js / NestJS / PostgreSQL / GCP を前提

理由：

- TTL とアトミックな失効が Redis で自然に表現でき、掃除ジョブが不要
- 主データは Cloud SQL(PostgreSQL) に集約し、揮発データは分離したい

結果：

- password-reset 等の Unit は Redis 前提で設計する(design が従う)
- Memystore の運用コスト・監視が Operations に加わる

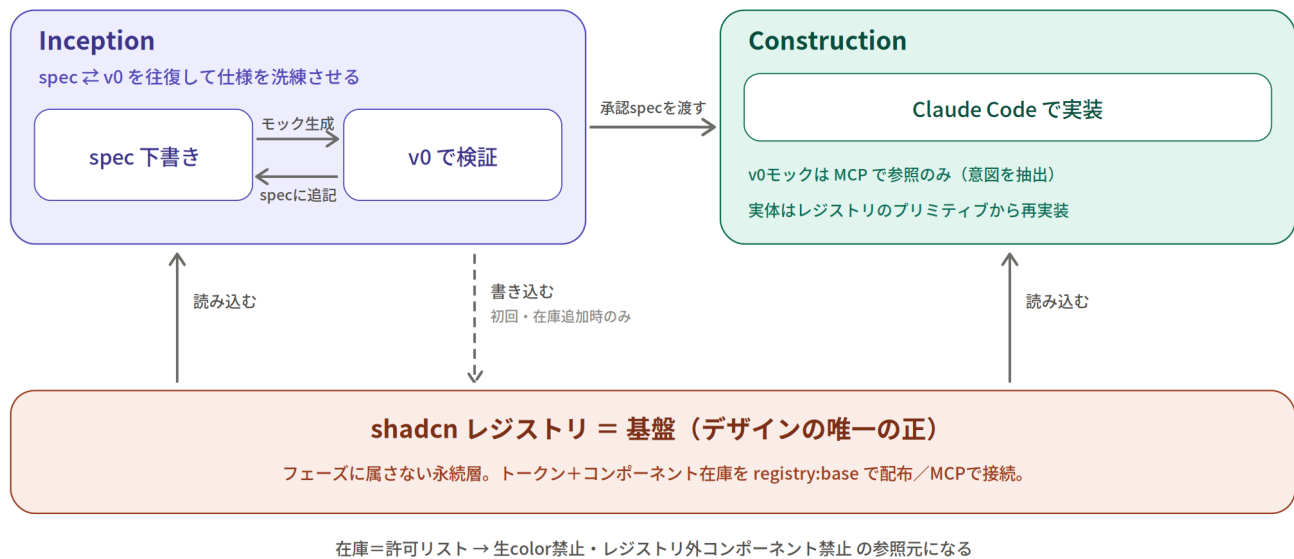
却下案：

- Cloud SQL に tokens テーブル：TTL 掃除とロック競合の運用負荷が高い
- JWT でサーバ状態なし：発行後に即時失効できず要件(30分)に反する

※ 採用スタック(Next.js / NestJS / PostgreSQL / GCP)のような大きな前提も ADR として残す。design (Unit内部の形) は ADR の決定に従うが、ADR 自体は特定の Unit に属さない。

補足：UI/UX統合 — デザインの二層と基盤レジストリ

デザインの正はコードに置く。デザインシステム(トークン・コンポーネント在庫)は施策に属さない常設の基盤で、ADR・docs/product/と同じ「フェーズに属さない永続層」として扱う。



エージェントが従う優先順位 (CLAUDE.mdに明記)

既存コードベース	> レジストリ	> モック (v0)
最優先	デザインの正	非正・意図の参照のみ

運用ルール (UI/UX) :

- ①デザインの決定もコードで保存する
- ②トークン=コード、コンポーネント=レジストリ
- ③視覚ツール (Figma) に正を作らない (AIが読めない並行した正を作らないため)
- ④featureをまたぐUIは design-system/components/ へ昇格 (Issue起票→設計承認を厚く)
- ⑤単一施策のUIは features/<施策>/components/ に留める

一貫性は機械強制 (目視レビューではない) : 色=トークンlint / 構造=コンポーネント来歴ゲート / 見た目=スクリーンショット回帰。Hooks・lint・ビジュアル回帰で「在庫=許可リスト」を強制し、生color・レジストリ外コンポーネントは物理的に混入できない状態にする。

永続コンテキスト基盤（全フェーズが読み書き）
spec・ADR・計画・テスト・ポストモテムをリポジトリで管理

```
repo/
├── CLAUDE.md # 常時コンテキスト(lean)：要約 + product/ へのポインタ
├── .mcp.json # MCP接続(shadcn・Playwright・DB…)
├── .claude/skills/ # AI-DLCプロセス系(施策横断)・commands・hooks
├── docs/ # ★永続コンテキスト
│   ├── product/ # 施策の上位、常設のプロダクト前提
│   │   ├── product.md # 面(surfaces)一覧・主要アクター/ロール・施策マップ
│   │   └── glossary.md # ユビキタス言語／用語集
│   ├── adr/NNNN.md # 不可逆な決定…決定ごとに1件
│   ├── auth/ # 施策(feature)
│   │   ├── intent.md # 狙い・KPI(ステップ1)…施策に1つ
│   │   ├── spec.md # ← ここに「対象面」を1行明記 …Intentに1つ
│   │   ├── overview.md # 分解マップ(ステップ4)…施策に1つ
│   │   └── password-reset/ # Unit(unit)を親にする
│   │       ├── plan.md # Unit実行プラン(ステップ5)…Unitごとに1つ
│   │       └── design.md # Unit詳細設計(ステップ5)…Unitごとに1つ
│   └── postmortems/ # 障害の学び(ステップ11)…件ごとに1つ
├── web/
│   ├── .claude/skills/ # デザインシステム・shadcn規約
│   ├── design-system/ # ★デザインシステム（フェーズに属さない永続層）
│   │   ├── tokens/ # ①トークンの源泉
│   │   │   ├── tokens.css # CSS変数(色・余白・タイポ・角丸・影)
│   │   │   └── tailwind.preset.ts # Tailwindテーマに写像(生color禁止lintの参照元)
│   │   ├── ui/ # ②shadcnプリミティブ在庫＝許可リスト
│   │   │   ├── button.tsx / input.tsx / dialog.tsx …
│   │   │   └── (CLIが追加する素のprimitive。ここは手で盛らない)
│   │   ├── components/ # ③bulletproof の components
│   │   │   ├── data-table.tsx / page-header.tsx / empty-state.tsx …
│   │   │   └── (Unitはまずここを使う。無ければ在庫追加をIssueで起票)
│   │   ├── registry/ # 配布定義(registry:base)
│   │   │   └── registry.json # ①②③を単一ペイロードとして配る宣言
│   │   ├── inventory.md # 在庫台帳(どのprimitive/合成物があるか)
│   │   └── design-system.md # 規約・原則(採用理由・do/don't・トークン設計方針)
│   └── features/<施策>/
│       └── components/ # 施策固有UI(昇格しない)
├── backend/
│   └── .claude/skills/ # API・DB系
├── .github/workflows/ # CI/CD+Claude Code Action(ステップ9)
└── tests/ # テスト設計・E2E(ステップ8)
```

運用ルール：

- ①各フェーズのAI成果物は必ずここに保存する(セッションが消えても文脈が残る)
- ②相互参照する(Issue→spec→ADRがリンクで辿れる)
- ③「AIが読めない場所の決定」を作らない

spec / overview / plan / design の違い

ファイル	役割・目的	記述される主な内容
spec.md Intentに1つ	仕様 (Specification) 「何を作るか(WHAT)」と「なぜ作るか(WHY)」	要件・目標・満たすべき条件(受け入れ基準)。実装方法(How)には触れない。
overview.md 施策に1つ	分解マップ (Overview) specをN個のUnitにどう割るか	Unit一覧・依存関係・実施順序・共有コンポーネントの洗い出し。施策全体を俯瞰する1枚。
plan.md Unitごとに1つ	Unit実行プラン (Plan) このUnitをどの順で作業するか	着手前の段取り(作業手順と検証順)。使用技術はADR、実装の中身はdesignに委ねる。
design.md Unitごとに1つ	設計 (Design) システムの内部構造を図解・明文化	Unit内部の構造:コンポーネント・シーケンス図・データフロー・ディレクトリ構成・API設計。施策全体を跨ぐ決定はADRへ。

導入ロードマップ（90日目安）

Day 1-30	土台	検証基盤の強化(テスト・CI・型)／CLAUDE.md・spec/ADRテンプレート整備／パイロットチームと対象Unitの選定
Day 31-60	ループ構築	Issue駆動パイプライン(ゲート③周り)を構築／AI一次レビュー導入／Mob Elaborationを週次で試行
Day 61-90	拡張と標準化	QA・CI修復・監視へ拡張／デプロイ・リリース管理を自動化(フラグ・カナリア)／DORAメトリクスで効果計測／標準化し横展開

参考:AWS「AI-Driven Development Lifecycle(AI-DLC)」ホワイトペーパー／awslabs/aidlc-workflows(GitHub)